



Instituto Superior de Engenharia

Politécnico de Coimbra

DEPARTAMENTO DE / DEPARTMENT OF
INFORMÁTICA E SISTEMAS

Trabalho Prático - Programação em C para UNIX Sistemas Operativos

Autor / Author

David Martins Correia – 2023140410

Tiago Gouveia Filipe – 2019112767



INSTITUTO POLITÉCNICO DE
COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Coimbra, 14 dezembro de 2024

ÍNDICE

Introdução.....	3
Estruturas de Dados e Constantes	4
Arquitetura	5
Servidor	5
Cliente.....	5
Comunicação	5
Gestão de <i>Threads</i> e acesso aos dados	6
Conclusão.....	7

Índice de Figuras

Figura 1 – Esquema de funcionamento	3
Figura 2 - Estrutura principal de dados utilizada	4
Figura 3 - Funcionamento dos mutexs	6

Introdução

Este relatório descreve o funcionamento, as implementações e os desafios técnicos de um sistema cliente-servidor que utiliza FIFOs (*First In, First Out*) para a comunicação. O sistema permite a criação, subscrição, envio de mensagens e gestão de tópicos, por parte do administrador operações, como por exemplo bloquear tópicos, gerir utilizadores e verificar mensagens.

O sistema é composto por:

Servidor: Gestão de tópicos, mensagens e utilizadores.

Cliente: Interage com o servidor através de comandos pré-definidos.

FIFOs: Canais de comunicação, entre cliente e servidor, exclusivos para cada cliente.

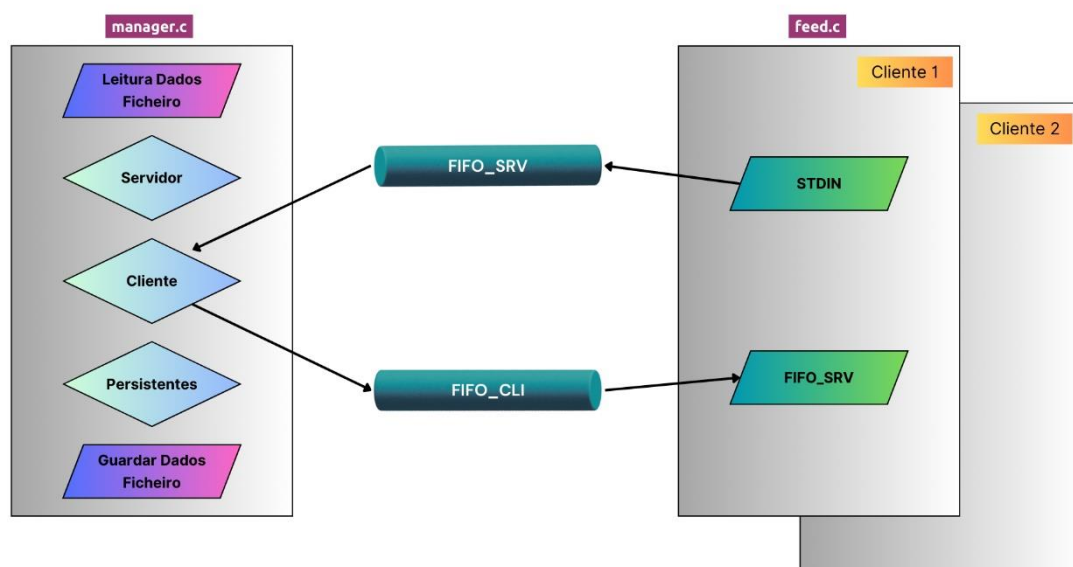


Figura 1 – Esquema de funcionamento

Estruturas de Dados e Constantes

As estruturas de dados, constantes e bibliotecas são definidas no ficheiro *utils.h* e são partilhadas entre cliente e servidor.

As estruturas estão definidas entre modelos base e modelos envelope que permitem facilitar na comunicação através de um valor numérico.

Para a implementar as funcionalidades pretendidas através dos comandos pré-definidos utilizamos a seguinte estrutura **CDATA**, para guardar utilizadores com sessão iniciada, nome dos tópicos e quantidade de tópicos atuais, bem como um contador para utilizadores, um *array* do tipo **TOPICOS**, onde fica registado o nome do tópico, os utilizadores subscritos a este tópico, o bloqueio do tópico, a quantidade de mensagens relativamente ao tópico e o *pid* de cada utilizador para facilitar no envio de mensagens. A variável *exit* é utilizada para o controlo das *threads*, explicado no capítulo referente as *Threads*.

```
typedef struct
{
    char nome[MAX_NAME_TOPICS], username[MAX_USERS][50];
    int bloqueio, count, pid[MAX_USERS];
    Mensagem msgs[MAX_MSG_PERSISTENTES];
} TOPICOS;

typedef struct
{
    char (*users)[50], (*topics)[MAX_NAME_TOPICS];
    int *user_atual, *topics_atual, *pid, exit;
    pthread_mutex_t *ptrinco;
    TOPICOS *pdados;
} CDATA;
```

Figura 2 - Estrutura principal de dados utilizada

Arquitetura

O sistema foi projetado em três componentes, Servidor (*manager.c*), Cliente (*feed.c*), e informações utilizadas nos dois ficheiros anteriores (*utils.h*).

Servidor

- Inicialização das estruturas partilhadas e os FIFOs necessários.
- Gestão dos tópicos, mensagens e subscrições.
- Utilização dos *threads* para o correto funcionamento do servidor tendo em conta o exigido.
 - *Thread* Servidor: recebe comandos através do *stdin*.
 - *Thread* Cliente: recebe comandos através do FIFO do servidor (**FIFO_SRV**) e nos casos necessários envia *outputs* para o ecrã do cliente através do FIFO do cliente (**f_pid**).
 - *Thread* Persistentes: gere o tempo de mensagens persistentes, tempo superior a zero, decrementando o tempo em uma unidade a cada segundo.

Cliente

- Permite a interação do utilizador com o servidor através comandos como *msg*, *subscribe* e *topics*.
- Utilização de ***select***, para controlo do sistema entre comandos enviados através do *stdin* e receção de mensagens por parte do servidor.

Comunicação

- FIFOs exclusivos para cada cliente (**FIFO_CLI**).
- Um FIFO central para o servidor (**FIFO_SRV**).

Gestão de *Threads* e acesso aos dados

Para o correto encerramento das *threads* foi utilizado, ***pthread_exit(NULL)***, para sair da *thread* corretamente. As *threads* de cliente e persistentes estão a trabalhar em função de uma *flag*, variável ***exit***, que é alterada após o término da *thread* servidor.

O comando ***close*** é usado para finalizar a *thread* do servidor. Se o servidor não tiver recebido nenhuma ligação por parte de um cliente, uma interação é simulada, enviando um valor para o FIFO do servidor e permitindo que o servidor seja fechado corretamente.

Com a análise verificou-se a exceção de terminar o servidor sem ter recebido nenhuma ligação por parte de um cliente. Esta situação foi tratada enviando um valor para o seu próprio FIFO (***FIFO_SRV***), através da simulação uma interação com um cliente, utilizando um valor definido para o encerramento da *thread* cliente.

Para a gestão do acesso aos dados foi utilizado ***mutexs***, ***pthread_mutex_t *ptrinco***; para este efeito. Com este método pretendemos restringir o acesso aos dados partilhados apenas num instante, evitando múltiplos acessos em simultâneo aos mesmo dados causando incoerência de dados. Pretendemos criar uma secção critica de acesso aos dados como no seguinte exemplo:

```
pthread_mutex_init(&trincos, NULL); //inicialização
pthread_mutex_lock(psr->ptrinco); //inicia a secção critica
//secção critica
pthread_mutex_unlock(psr->ptrinco); //encerra a secção critica
pthread_mutex_destroy(&trincos); //destruição
```

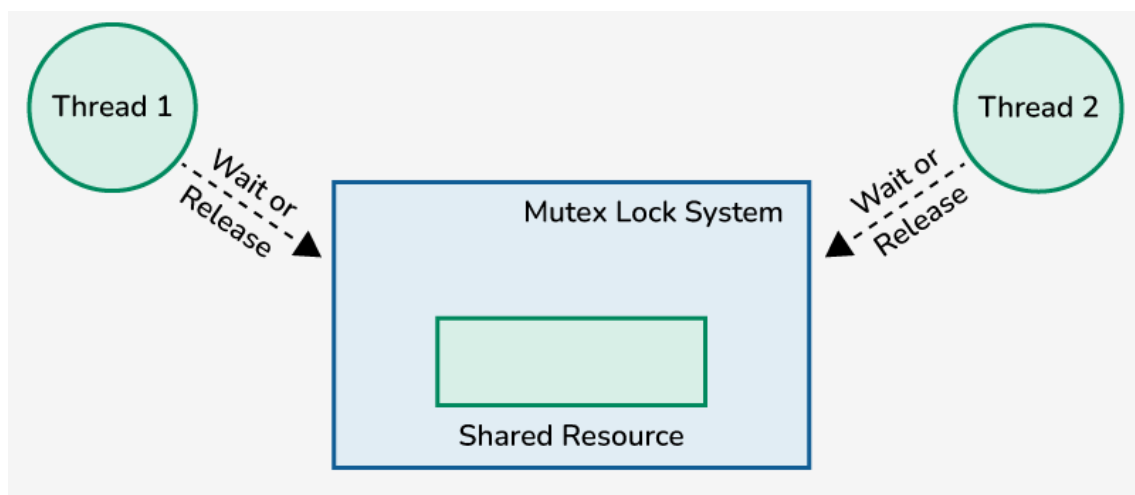


Figura 3 - Funcionamento dos mutexs

Conclusão

O trabalho desenvolvido não alcançou todos os objetivos propostos. As estruturas de dados e o modelo de comunicação foram projetados para garantir consistência e desempenho tendo sido alteradas ao longo da execução do programa consoante as necessidades.

As estruturas de dados foram desenhadas de forma a garantir consistência e integridade das informações partilhadas entre *threads* e processos. A utilização de **FIFOs** e *mutexes* revelou-se essencial para assegurar uma comunicação eficiente e o controlo de acesso aos recursos partilhados, apesar de não ter sido corretamente implementados ao longo de todo o trabalho.

Entre os principais desafios enfrentados, destacam-se a gestão correta das *threads* e o controle da sincronização. No entanto, o sistema conseguiu oferecer funcionalidades robustas como a criação e gestão de tópicos, envio de mensagens e interação dinâmica entre cliente e servidor.